

Simulazione della Diffusione del Calore 2D

PROGETTO DI PRINCIPI DELLA
PROGRAMMAZIONE PARALLELA
PROFF. SCIACCA E VITELLO

Equazione del Calore 2D

- L'idea di questo progetto è quello di simulare la diffusione del calore su una griglia N x N sottoposta a diverse fonti di calore che provengono dai lati della griglia a diverse temperature.
- Il calcolo del calore per ogni punto è dato dalla seguente equazione:

$$u_{new}[i][j] = (u[i - 1][j] + u[i + 1][j] + u[i][j - 1] + u[i][j + 1]) / 4.0$$

Equazione del calore 2D

u	0	1	2	3	4	5	6	7	8	9
0										
1										
2										
3										
4					6					
5				4	10	8				
6					12					
7										
8										
9										

u_{new}	0	1	2	3	4	5	6	7	8	9
0										
1										
2										
3										
4										
5					?					
6										
7										
8										
9										

$$u_{new}[5][4] = \frac{(u[4][4] + u[6][4] + u[5][3] + u[5][5])}{4.0} = \frac{(4 + 6 + 8 + 12)}{4.0} = 7.5$$

Equazione del calore 2D

- La complessità di una singola iterazione del do-while è $O(N^2)$ a causa dei due cicli for annidati.
- Il do-while viene eseguito un numero K di volte che cresce all'aumentare non solo di N ma anche in base al valore di convergenza impostato. Di fatto, ciò rende K un valore assolutamente non trascurabile che, dai dati raccolti sembra essere circa un $O(N^2)$.
- Di fatto dunque la risoluzione ha un costo $O(K \cdot N^2)$ che è circa $O(N^4)$.

```
Inizializza matrici u e u_new (N x N) con bordi di Dirichlet  
eps_sq = EPS * EPS  
iterazioni = 0
```

```
DO
```

```
    differenza = 0.0
```

```
    // Scansione del dominio interno
```

```
    FOR i = 1 TO N - 1
```

```
        FOR j = 1 TO N - 1
```

```
            // Aggiornamento tramite stencil a 5 punti
```

```
            u_new[i, j] = 0.25 * (u[i-1, j] + u[i+1, j] +  
                                u[i, j-1] + u[i, j+1])
```

```
            // Calcolo scarto per la convergenza L2
```

```
            tmp = u_new[i, j] - u[i, j]  
            differenza = differenza + (tmp * tmp)
```

```
        END FOR
```

```
    END FOR
```

```
    Scambia(u, u_new)
```

```
    iterazioni = iterazioni + 1
```

```
WHILE (differenza > eps_sq)
```

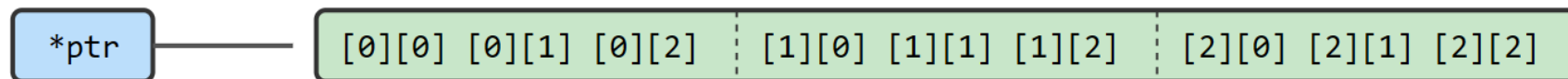
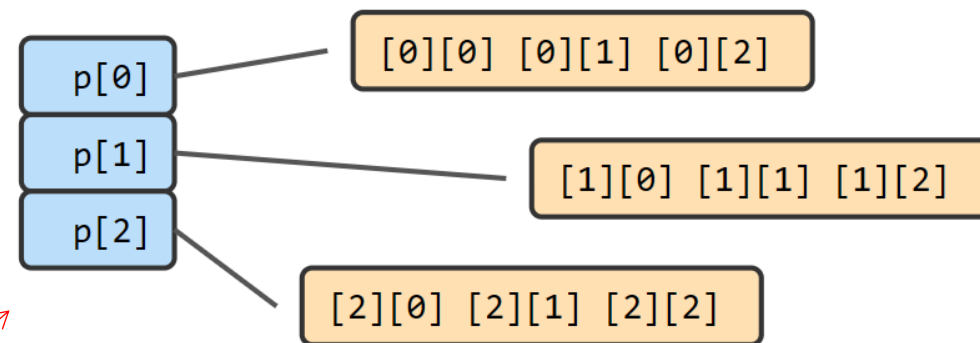


Hardware di Test

- **Processore:** AMD Ryzen 5 5500U (Architettura Zen 2 / Lucienne, x86_64)
- **Topologia Core:** 6 Core Fisici / 12 Core Logici (Thread per core: 2, SMT abilitato)
- **Frequenza di Clock:** Base 2.10 GHz / Max Boost 4.05 GHz
- **Sottosistema di Cache:**
 - Cache L1d: 192 KiB totali
 - Cache L2: 3 MiB totali
 - Cache L3: 8 MiB totali
- **Memoria RAM:** 8 GB DDR4 (Memoria disponibile a sistema: ~5.8 GB unificati)

Codice Sequenziale – Inizializzazione

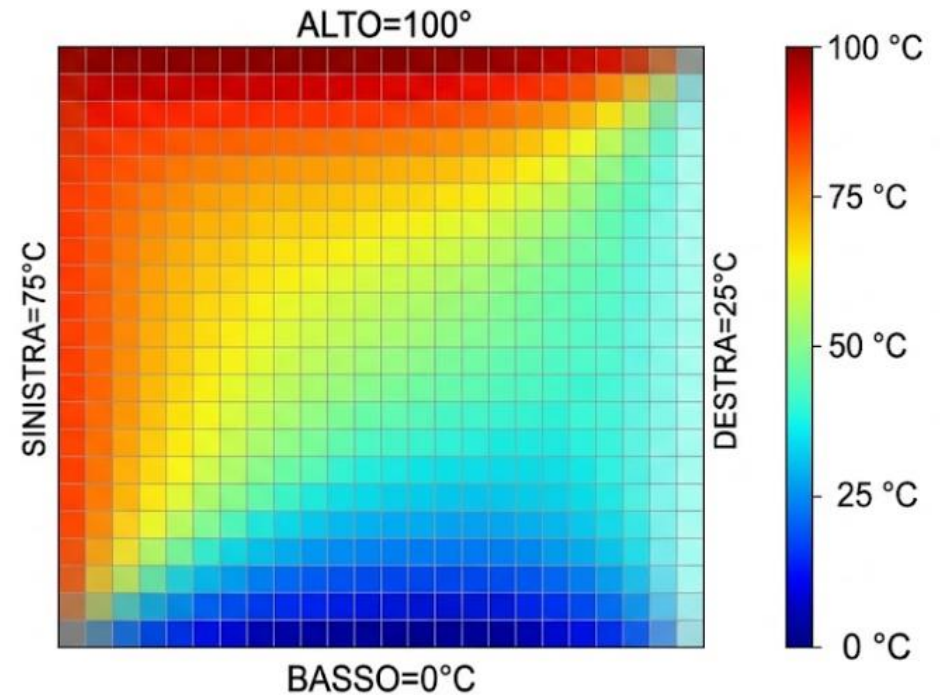
- Per inizializzare la matrice, si avevano principalmente due scelte:
 - **Inizializzazione con Calloc multipla:** chiamare più volte la calloc, genera più sequenze di memorie contigue ma che, tra di loro, non lo sono, rischiando di avere un maggior numero di cache miss.
 - **Inizializzazione con Calloc singola:** utilizzare una singola calloc fa sì che otteniamo una sola sequenza di memoria contigua che siamo, però, costretti a gestire come una matrice attraverso l'aritmetica dei puntatori.



Codice sequenziale – Inizializzazione

```
for (int i = 1; i < N - 1; i++) {  
    u[0 * N + i] = u_new[0 * N + i] = TOP;  
    u[(N - 1) * N + i] = u_new[(N - 1) * N + i] = BOT;  
    u[i * N + 0] = u_new[i * N + 0] = LEFT;  
    u[i * N + (N - 1)] = u_new[i * N + (N - 1)] = RIGHT;  
}
```

```
u[0 * N + 0] = u_new[0 * N + 0] = TOP;  
u[0 * N + (N - 1)] = u_new[0 * N + (N - 1)] = RIGHT;  
u[(N - 1) * N + 0] = u_new[(N - 1) * N + 0] = LEFT;  
u[(N - 1) * N + (N - 1)] = u_new[(N - 1) * N + (N - 1)] = BOT;
```



Codice sequenziale – Calcolo

```
do
{
    differenza = 0.0;

    for (int i = 1; i < N-1; i++)
    {
        for (int j = 1; j < N-1; j++)
        {
            int idx_sopra = ((i-1) * N) + j;
            int idx_sotto = ((i+1) * N) + j;
            int idx_sx = ((i * N) + j) - 1;
            int idx_dx = ((i * N) + j) + 1;

            u_new[(i*N)+j] = (u[idx_sopra] + u[idx_sotto] + u[idx_sx] + u[idx_dx]) * 0.25;

            double tmp_val = u_new[(i*N)+j] - u[(i*N)+j];
            differenza += (tmp_val * tmp_val);
        }
    }

    double *tmp_ptr = u;
    u = u_new;
    u_new = tmp_ptr;

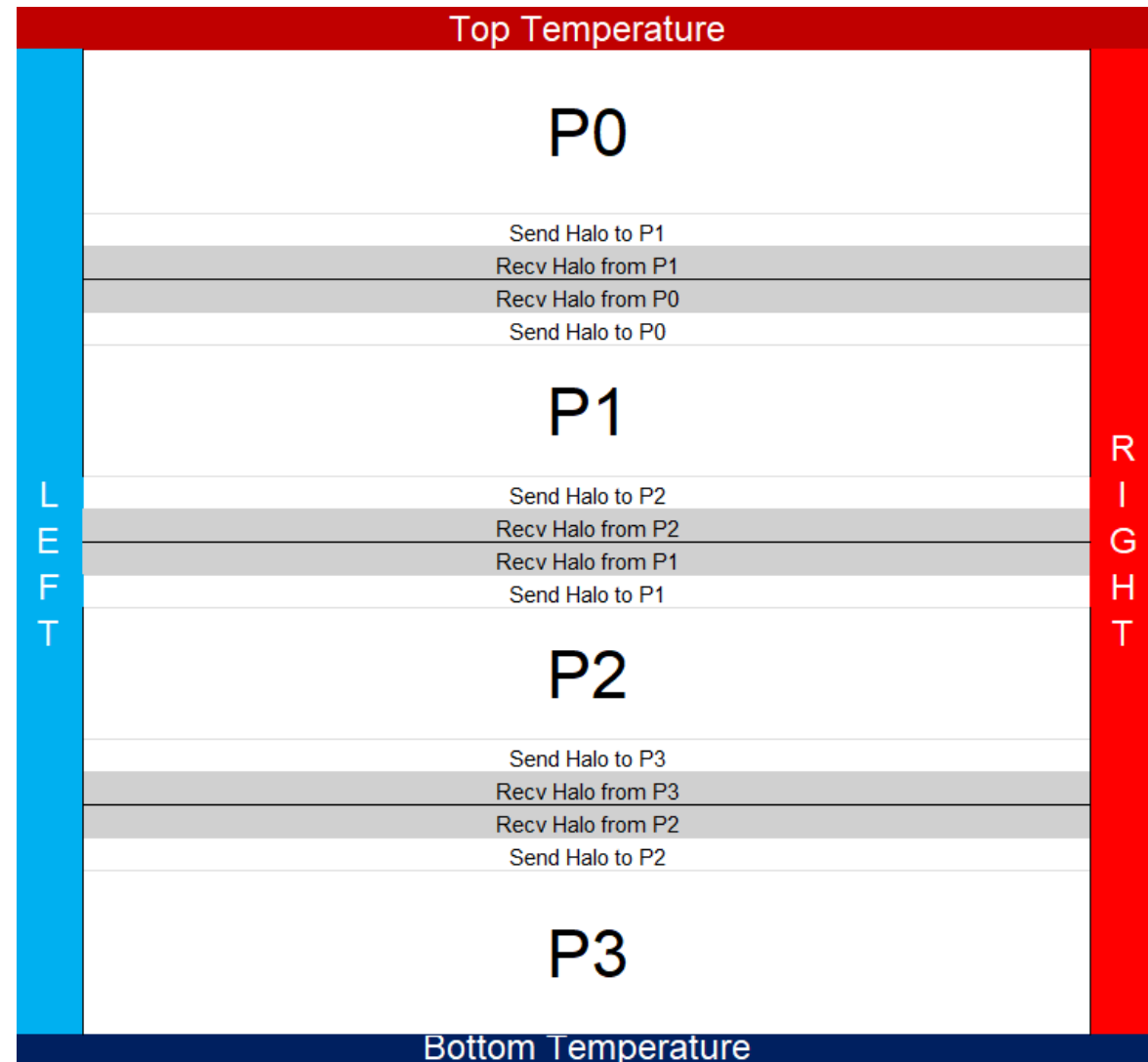
    iterazioni++;
} while (differenza > eps_sq);
```

- Il codice sequenziale, banalmente, ricalca alla perfezione quello che è lo pseudocodice visto precedentemente.

MPI- Topologia

La matrice viene divisa secondo un partizionamento orizzontale cercando di assegnare ad ogni processo un **numero equo di righe da calcolare**.

Grazie all'uso di `MPI_Cart_create`, è possibile effettuare questo tipo di decomposizione molto facilmente gestendo facilmente l'identificazione e lo scambio di messaggi con i vicini tramite le **righe di Halo**.



MPI – Latency Hiding

- Una volta identificati i vicini tramite la `MPI_Cart_shift`, la prima cosa che facciamo dentro il do-while è proprio quella di inviare e ricevere gli **Halo** tramite chiamate **asincrone**, così ch , in attesa del completamento dello scambio, cominciamo ad effettuare i calcoli necessari, di fatto, riducendo l'impatto che ha la comunicazione sull'intero codice.

```
MPI_Comm_rank(comm_cart, &rank);
```

```
int rank_up, rank_down;
```

```
MPI_Cart_shift(comm_cart, 0,1,&rank_up,&rank_down);
```

```
MPI_Request req_send[2];
```

```
MPI_Request req_recv[2];
```

```
MPI_Irecv(&u[0], N, MPI_DOUBLE, rank_up, 0, comm_cart, &req_recv[0]);
```

```
MPI_Irecv(&u[(righe_per_processo+1) * N], N, MPI_DOUBLE, rank_down, 0, comm_cart, &req_recv[1]);
```

```
MPI_Isend(&u[1* N], N, MPI_DOUBLE, rank_up, 0, comm_cart, &req_send[0]);
```

```
MPI_Isend(&u[(righe_per_processo * N)], N, MPI_DOUBLE, rank_down, 0, comm_cart, &req_send[1]);
```

MPI – Calcolo

Il calcolo interno in questa versione viene fatto senza utilizzo di ulteriori Thread, lavorando esclusivamente in sequenziale. Tutte le righe vengono calcolate normalmente meno che la prima e l'ultima che verranno calcolate successivamente dopo che si è certi che si è ricevuto l'Halo dai vicini.

```
for (int i = 2; i < righe_per_processo; i++)
{
    for (int j = 1; j < N-1; j++)
    {
        int idx_sopra = ((i-1) * N) + j;
        int idx_sotto = ((i+1) * N) + j;
        int idx_sx = ((i * N) + j) - 1;
        int idx_dx = ((i * N) + j) + 1;

        u_new[(i*N)+j] = (u[idx_sopra] + u[idx_sotto] + u[idx_sx] + u[idx_dx]) * 0.25;

        double tmp = u_new[(i*N)+j] - u[(i*N)+j];
        differenza += (tmp * tmp);
    }
}
```

MPI – Calcolo – Waitall

```
MPI_Waitall(2, req_recv, MPI_STATUS_IGNORE);

for (int i = 1; i < N-1; i++) //calcoliamo le ultime righe arrivate
{
    // Solo chi non è il primo processo calcola la prima riga dato che lui può effettivamente calcolarla a
    // prescindere dato che non ha bisogno dello scambio
    if (rank != 0) {
        u_new[N+i] = (u[i] + u[N+i-1] + u[N+i+1]+u[(2*N)+i]) * 0.25;
        double tmp_1 = u_new[N+i] - u[N+i];
        differenza += (tmp_1 * tmp_1);
    }

    // Solo chi non è l'ultimo processo calcola l'ultima riga per le stesse ragioni descritte sopra
    if (rank != num_processi - 1) {
        u_new[(righe_per_processo * N)+i] = (u[(righe_per_processo-1) * N+i] + u[(righe_per_processo * N)
        +i -1] + u[(righe_per_processo * N)+i +1] + u[(righe_per_processo+1) * N+i]) * 0.25;
        double tmp_2 = u_new[(righe_per_processo * N)+i] - u[(righe_per_processo * N)+i];
        differenza += (tmp_2 * tmp_2);
    }
}

MPI_Waitall(2, req_send, MPI_STATUS_IGNORE);
MPI_Allreduce(&differenza, &differenza_globale, 1, MPI_DOUBLE, MPI_SUM, comm_cart);

double *tmp_swap = u;
u = u_new;
u_new = tmp_swap;

iterazioni++;

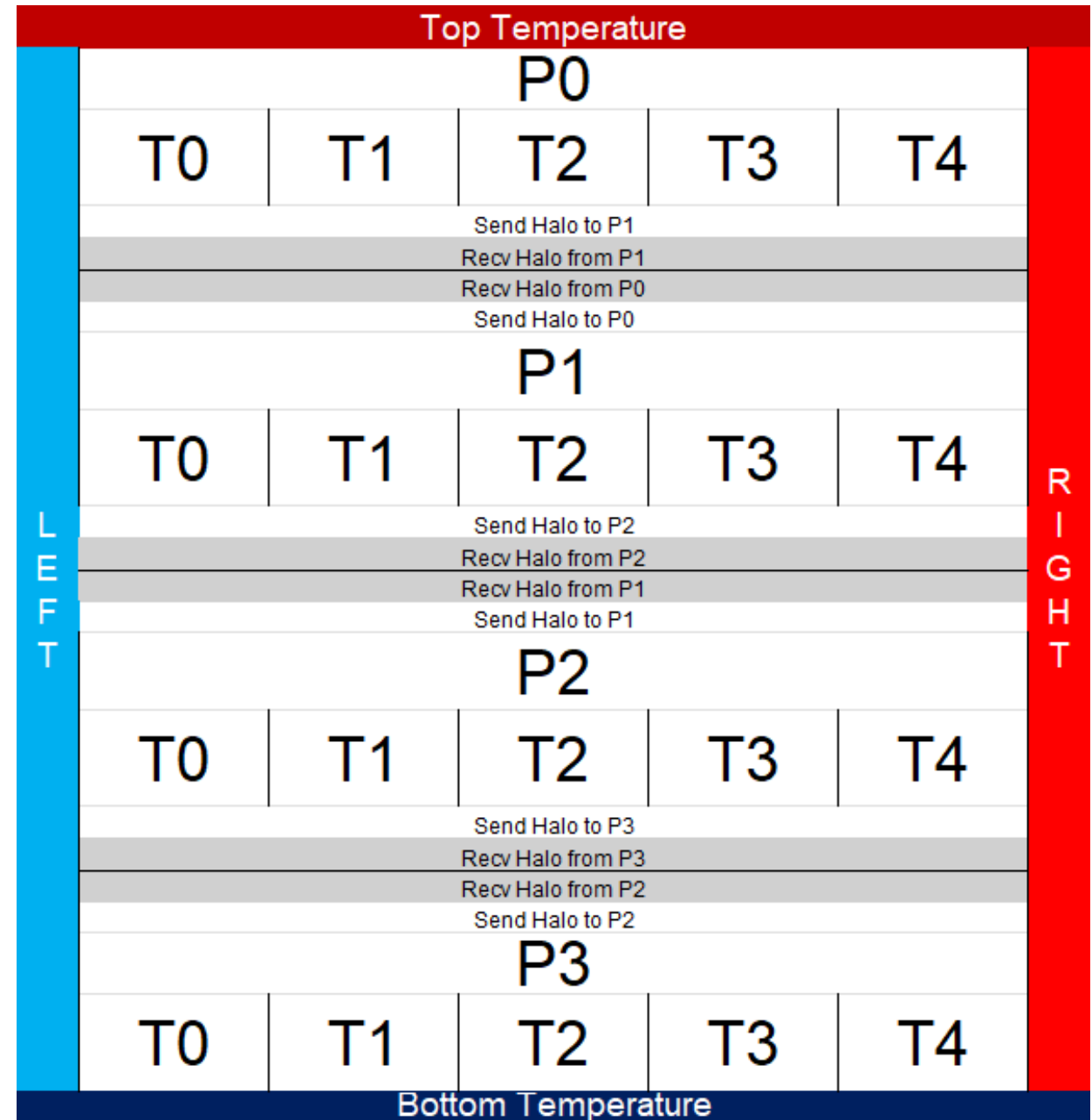
} while (differenza_globale > eps sa):
```

- A questo punto, una volta certi che abbiamo completato lo scambio con tutti i processi, possiamo calcolare le righe mancanti.

MPI + OpenMP

Macro-partizionamento (MPI): Suddivide il dominio 2D in strisce orizzontali e gestisce lo scambio asincrono degli Halo (Ghost Cells) tra i nodi.

Micro-partizionamento (OpenMP): Parallelizza i cicli annidati per il calcolo dello stencil a 5 punti sfruttando tutti i core all'interno del singolo nodo.



MPI + OpenMP – Inizializzazione

Ottimizzazione NUMA: Implementazione della politica *First-Touch* in fase di inizializzazione per garantire la località dei dati.

Vantaggio: Previene il sovraccarico del bus di memoria, un problema tipico quando un singolo thread (master) pre-allocava l'intera struttura dati.

```
double *u = (double *) malloc(dimensione_totale * sizeof(double));
double *u_new = (double *) malloc(dimensione_totale * sizeof(double));

#pragma omp parallel for schedule(static)
for (int i = 0; i < righe_per_processo + 2; i++) {
    for (int j = 0; j < N; j++) {
        u[(i * N) + j] = 0.0;
        u_new[(i * N) + j] = 0.0;
    }
}
```

MPI + OpenMP – Calcolo

Il calcolo interno è parallelizzato tramite OpenMP (`#pragma omp parallel for`). Per evitare *data race* sulla variabile globale **differenza** – che causerebbe la perdita di aggiornamenti e corromperebbe la verifica di convergenza – è obbligatorio proteggere l'accumulo con la clausola `reduction(+:differenza)`.

```
#pragma omp parallel for reduction(+:differenza) schedule(static)
for (int i = 2; i < righe_per_processo; i++)
{
    for (int j = 1; j < N-1; j++)
    {
        int idx_sopra = ((i-1) * N) + j;
        int idx_sotto = ((i+1) * N) + j;
        int idx_sx = ((i * N) + j) - 1;
        int idx_dx = ((i * N) + j) + 1;

        u_new[(i*N)+j] = (u[idx_sopra] + u[idx_sotto] + u[idx_sx] + u[idx_dx]) * 0.25;

        double tmp = u_new[(i*N)+j] - u[(i*N)+j];
        differenza += (tmp * tmp);
    }
}
MPI_Waitall(2, req_recv, MPI_STATUS_IGNORE);

#pragma omp parallel for schedule(static) reduction(+:differenza)
for (int i = 1; i < N-1; i++)
{
    if (rank != 0) {
        u_new[N+i] = (u[i] + u[N+i-1] + u[N+i+1] + u[(2*N)+i]) * 0.25;
        double tmp_1 = u_new[N+i] - u[N+i];
        differenza += (tmp_1 * tmp_1);
    }

    if (rank != num_processi - 1) {
        u_new[(righe_per_processo * N)+i] = (u[((righe_per_processo-1) * N)+i] + u[(righe_per_processo * N)+i-1] + u[(righe_per_processo * N)+i+1] + u[((righe_per_processo+1) * N)+i]) * 0.25;
        double tmp_2 = u_new[(righe_per_processo * N)+i] - u[(righe_per_processo * N)+i];
        differenza += (tmp_2 * tmp_2);
    }
}
```

Analisi delle prestazioni

STRUMENTI UTILIZZATI:

- PERF
- CLOCK_GETTIME



Metrische di Analisi

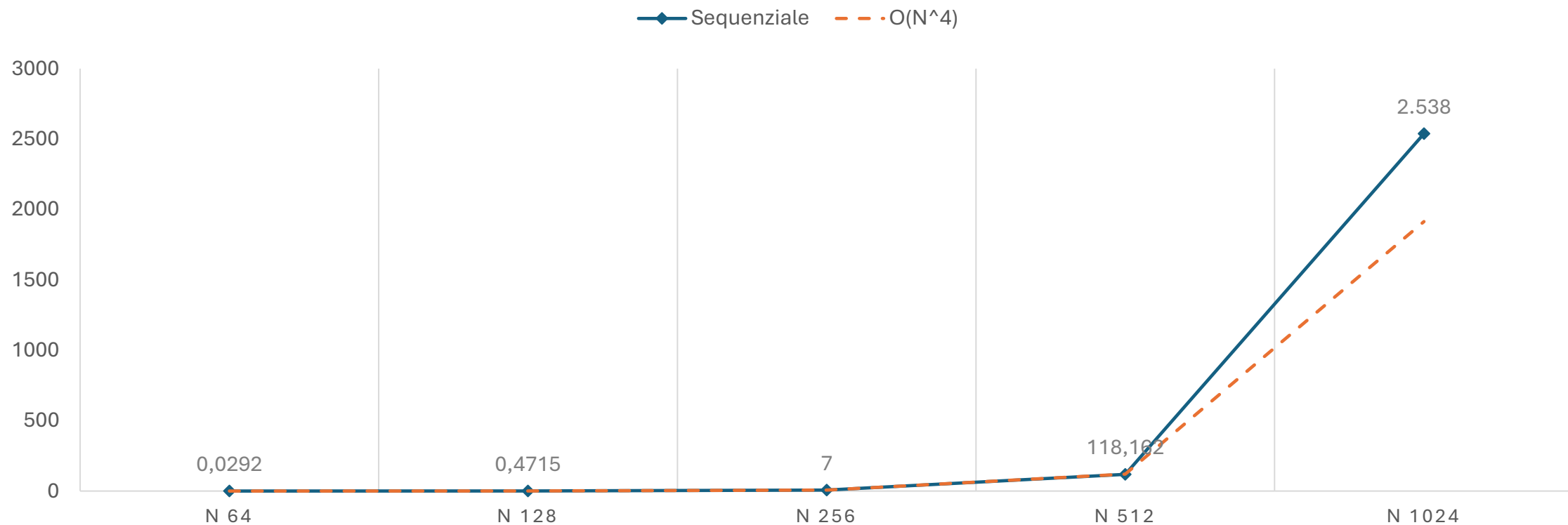
Per analizzare le prestazioni del codice, sono state considerate:

- Speedup = $S_p = \frac{T_1}{T_p}$
- Efficienza = $E_p = \frac{S_p}{p}$

Inoltre, sono stati considerati i seguenti setup per mettere a paragone MPI puro con la versione Ibrida:

- 8 Processi x 1 Thread
- 4 Processi x 2 Thread
- 2 Processi x 4 Thread
- 1 Processo x 8 Thread

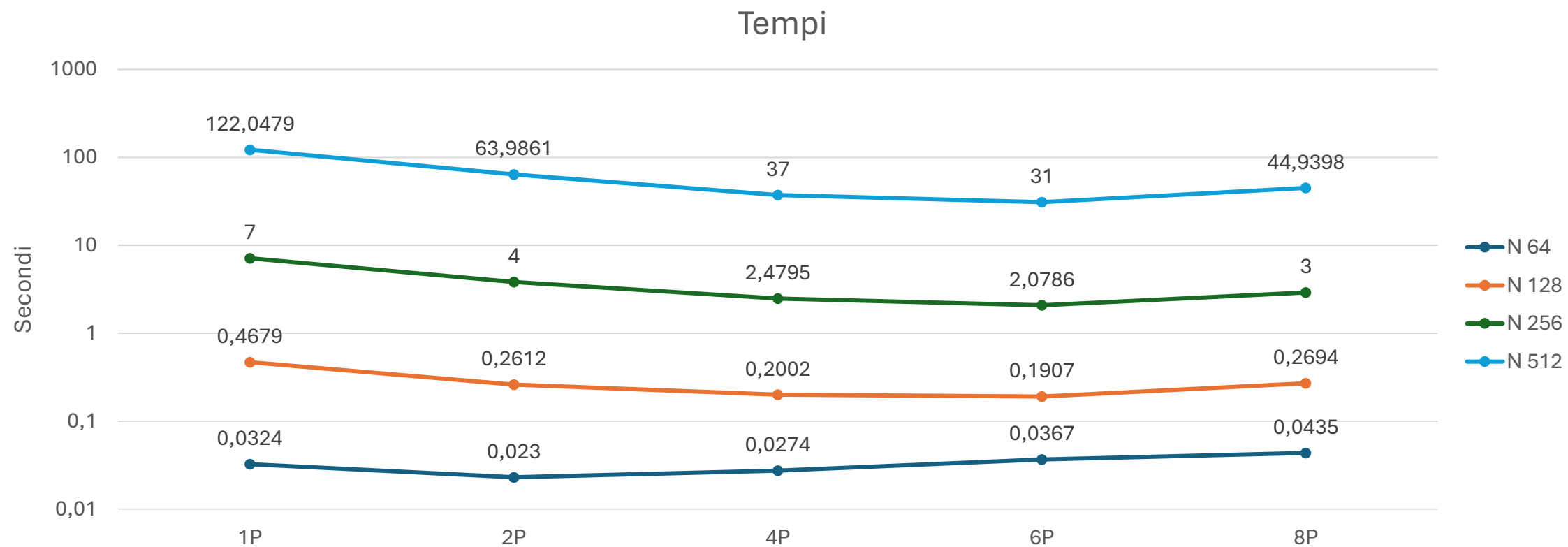
Analisi – Tempo Sequenziale



Analisi – Tempo Sequenziale

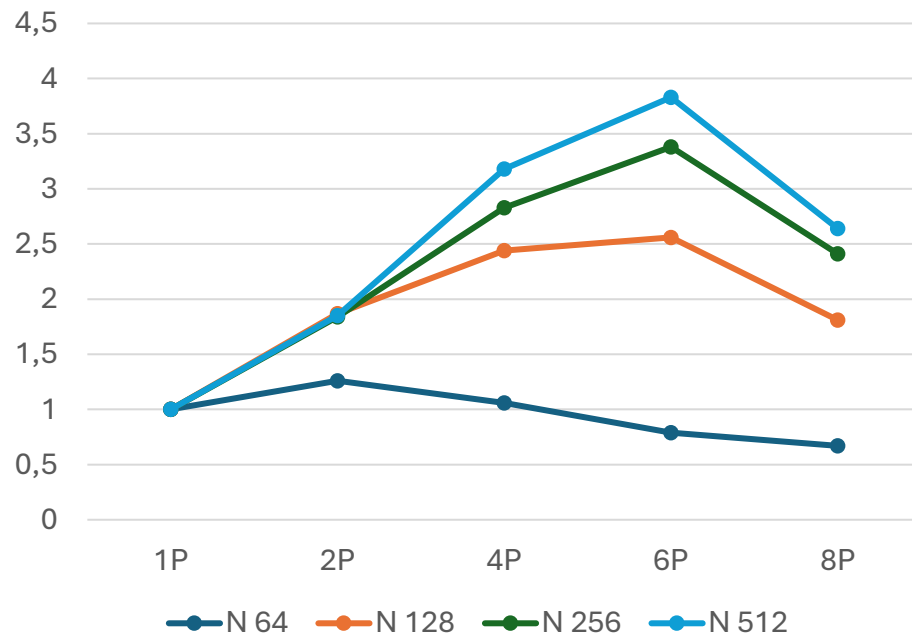
- A $N = 512$ il tempo reale ricalca fedelmente la curva teorica $O(N^4)$.
- A $N = 1024$, il tempo reale (2537.81 s) devia nettamente verso l'alto rispetto alla proiezione asintotica (1913.65 s). Perché?
- La **cache L3** ha uno spazio totale di **8MB** mentre, invece, le due matrici occupano uno spazio di circa **8 MB** l'una, rendendo quindi il totale **16 MB** forzando dunque continui accessi alla RAM.

Analisi – Tempo MPI

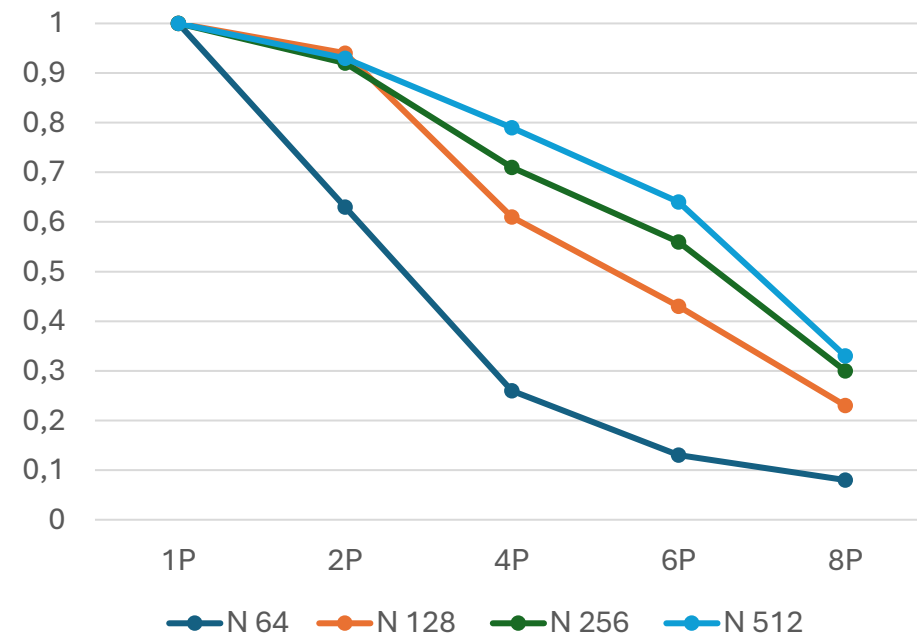


Analisi – Strong Scaling – MPI

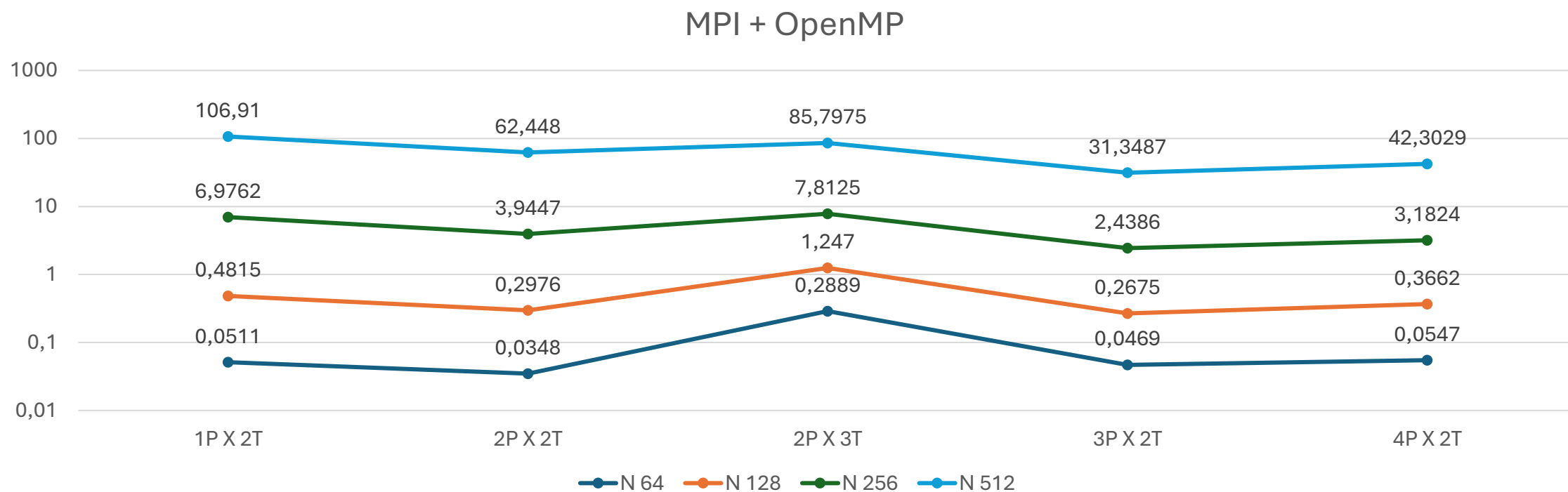
Speedup



Efficienza



Analisi – Tempo Ibrido



Analisi – Strong Scaling – Ibrido

